

✓ PERTEMUAN 7 - LINKED LIST

- Nama: Nadine Valia Azzahra
- NIM: 2C2230007
- PRODI: Sains Data

✓ STUDI KASUS PRAKTIKUM

```
# Studi Kasus 1 - Playlist Musik Digital

class Node:
    def __init__(self, lagu):
        self.lagu = lagu
        self.next = None

class Playlist:
    def __init__(self):
        self.head = None

    def tambah_lagu(self, lagu):
        node_baru = Node(lagu)

        if self.head is None:
            self.head = node_baru
            return

        current = self.head
        while current.next:
            current = current.next

        current.next = node_baru

    def tampilkan(self):
        current = self.head
        while current:
            print(current.lagu)
            current = current.next

playlist = Playlist()

playlist.tambah_lagu("The Man Who Can't Be Moved - The Script")
playlist.tambah_lagu("Merry Christmas, I Miss You - Alex Crichton")
playlist.tambah_lagu("Merry Christmas, Please Don't Call - Bleachers")
```

```
print("Daftar Lagu:")  
playlist.tampilkan()
```

```
Daftar Lagu:  
The Man Who Can't Be Moved - The Script  
Merry Christmas, I Miss You - Alex Crichton  
Merry Christmas, Please Don't Call - Bleachers
```

✓ Analisis:

Program menggunakan Linked List untuk menyimpan daftar lagu. Lagu baru dapat ditambahkan secara dinamis tanpa menentukan kapasitas terlebih dahulu. Setiap lagu direpresentasikan sebagai node yang saling terhubung.

```
# Studi Kasus 2 - Browser History  
  
class Node:  
    def __init__(self, halaman):  
        self.halaman = halaman  
        self.next = None  
  
class BrowserHistory:  
    def __init__(self):  
        self.head = None  
  
    def kunjungi(self, halaman):  
        node_baru = Node(halaman)  
  
        if self.head is None:  
            self.head = node_baru  
            return  
  
        current = self.head  
        while current.next:  
            current = current.next  
  
        current.next = node_baru  
  
    def tampilkan(self):  
        current = self.head  
  
        print("Riwayat Browser:")  
        while current:  
            print(current.halaman)  
            current = current.next  
  
history = BrowserHistory()
```

```
history.kunjungi("google.com")
history.kunjungi("youtube.com")
history.kunjungi("github.com")

history.tampilkan()
```

```
Riwayat Browser:
google.com
youtube.com
github.com
```

✓ Analisis:

Program menyimpan riwayat kunjungan website menggunakan Linked List. Setiap halaman yang dikunjungi ditambahkan pada akhir Linked List sehingga urutan kunjungan tetap terjaga.

```
# Studi Kasus 3 - Daftar Anggota Koperasi

class Node:
    def __init__(self, nama):
        self.nama = nama
        self.next = None

class Koperasi:
    def __init__(self):
        self.head = None

    def tambah_anggota(self, nama):
        node_baru = Node(nama)

        node_baru.next = self.head
        self.head = node_baru

    def hapus_anggota(self, nama):
        current = self.head

        if current and current.nama == nama:
            self.head = current.next
            return

        while current and current.next:
            if current.next.nama == nama:
                current.next = current.next.next
                return
            current = current.next

    def tampilkan(self):
        current = self.head
```

```

        while current:
            print(current.nama)
            current = current.next

koperasi = Koperasi()

koperasi.tambah_anggota("Tinky Winky")
koperasi.tambah_anggota("Dipsy")
koperasi.tambah_anggota("Asep")

koperasi.hapus_anggota("Asep")

print("Daftar Anggota:")
koperasi.tampilkan()

```

```

Daftar Anggota:
Dipsy
Tinky Winky

```

✓ Analisis:

Program memungkinkan data anggota ditambah dan dihapus kapan saja. Linked List cocok digunakan karena perubahan data dapat dilakukan tanpa menggeser elemen lain seperti pada Array.

```

# Studi Kasus 4 - Sistem Reservasi Hotel

class Node:
    def __init__(self, nama):
        self.nama = nama
        self.next = None

class ReservasiHotel:
    def __init__(self):
        self.head = None

    def tambah_reservasi(self, nama):
        node_baru = Node(nama)

        if self.head is None:
            self.head = node_baru
            return

        current = self.head

        while current.next:
            current = current.next

```

```

        current.next = node_baru

    def tampilkan(self):
        current = self.head

        print("Daftar Reservasi:")
        while current:
            print(current.nama)
            current = current.next

    hotel = ReservasiHotel()

    hotel.tambah_reservasi("Denis")
    hotel.tambah_reservasi("Fizi")
    hotel.tambah_reservasi("Tok Dalang")

    hotel.tampilkan()

```

```

Daftar Reservasi:
Denis
Fizi
Tok Dalang

```

✓ Analisis:

Program menyimpan data reservasi hotel dalam Linked List. Setiap reservasi baru ditambahkan secara dinamis sehingga jumlah data dapat berubah sesuai kebutuhan.

```

# Studi Kasus 5 - Sistem Antrian Rumah Sakit

class Node:
    def __init__(self, pasien):
        self.pasien = pasien
        self.next = None

class AntrianRS:
    def __init__(self):
        self.head = None

    def tambah_pasien(self, pasien):
        node_baru = Node(pasien)

        if self.head is None:
            self.head = node_baru
            return

        current = self.head

        while current.next:

```

```

        current = current.next

    current.next = node_baru

    def tampilkan(self):
        current = self.head

        print("Antrian Pasien:")
        while current:
            print(current.pasien)
            current = current.next

    antrian = AntrianRS()

    antrian.tambah_pasien("Pasien A")
    antrian.tambah_pasien("Pasien B")
    antrian.tambah_pasien("Pasien C")

    antrian.tampilkan()

```

```

Antrian Pasien:
Pasien A
Pasien B
Pasien C

```

✓ Analisis:

Program merepresentasikan sistem antrian pasien menggunakan Linked List. Pasien baru ditambahkan di bagian belakang antrian sehingga urutan pelayanan tetap sesuai prinsip FIFO (First In First Out).

```

# Studi Kasus 6 - Undo dan Redo Editor Teks (Doubly Linked List)

class Node:
    def __init__(self, aksi):
        self.aksi = aksi
        self.prev = None
        self.next = None

class Editor:
    def __init__(self):
        self.current = None

    def tambah_aksi(self, aksi):
        node_baru = Node(aksi)

        if self.current is None:
            self.current = node_baru
            return

```

```
        self.current.next = node_baru
        node_baru.prev = self.current
        self.current = node_baru

    def undo(self):
        if self.current and self.current.prev:
            self.current = self.current.prev

    def redo(self):
        if self.current and self.current.next:
            self.current = self.current.next

    def tampilkan(self):
        if self.current:
            print("Posisi Saat Ini:", self.current.aksi)

editor = Editor()

editor.tambah_aksi("Menulis Judul")
editor.tambah_aksi("Menulis Paragraf")
editor.tambah_aksi("Menambahkan Gambar")

editor.undo()
editor.tampilkan()

editor.redo()
editor.tampilkan()
```

```
Posisi Saat Ini: Menulis Paragraf
Posisi Saat Ini: Menambahkan Gambar
```

Analisis:

Program menggunakan Doubly Linked List karena setiap node memiliki pointer prev dan next. Struktur ini memungkinkan perpindahan maju (redo) dan mundur (undo) dengan mudah sehingga sangat cocok digunakan pada editor teks.

